



AI Memory Deletion Engine

Machine Unlearning Dashboard

A full-stack application demonstrating **Gradient Ascent Machine Unlearning** on Large Language Models. Fine-tune Qwen2.5-1.5B-Instruct on personal data, then selectively make it *forget* specific information with verified, real parameter updates.

Author: Akash Khurd

Repository: github.com/Akash1702-eng/Ai-Memory-Deletion-Engine

Technology: FastAPI · PyTorch · PEFT/LoRA · Qwen2.5-1.5B · Kaggle GPU

Date: August 2026

Table of Contents

01

Abstract

02

Introduction & Motivation

03

System Architecture

04

Project Structure

05

Core Algorithm — Gradient Ascent Unlearning

06

Backend — FastAPI Application

07

ML Pipeline — Training, Unlearning & Evaluation

08

Kaggle GPU Integration

09

Frontend — Dashboard SPA

10

Authentication & Subscription System

11

Payment Integration (Razorpay)

12

Email Service (SMTP/OTP)

13

Database Design (SQLite)

14

API Reference

15

Evaluation Metrics & MIA

16 End-to-End User Workflow

17 Technology Stack

18 Setup & Installation

19 References

1. Abstract

The **AI Memory Deletion Engine** is a full-stack application that demonstrates *genuine* machine unlearning on Large Language Models (LLMs). Unlike simple data deletion or prompt filtering, this project performs actual parameter updates using **Gradient Ascent** to make a fine-tuned model *forget* specific information it previously memorized.

The system uses **Qwen2.5-1.5B-Instruct** as the base model and applies **LoRA (Low-Rank Adaptation)** adapters for both fine-tuning and unlearning. Users upload personal Q&A data via CSV, fine-tune the model to memorize it, then selectively unlearn chosen memories. The entire pipeline is verified with before-after metric comparisons and a **Membership Inference Attack (MIA)** that statistically proves the data was genuinely forgotten.

GPU-intensive workloads (fine-tuning and gradient ascent unlearning) are offloaded to **Kaggle's free GPU** via the Kaggle API, while the lightweight FastAPI backend handles orchestration, chat inference, and the dashboard UI locally.

2. Introduction & Motivation

2.1 Problem Statement

With increasing concerns about data privacy (GDPR "Right to be Forgotten") and the memorization of sensitive information by LLMs, there is a critical need for mechanisms that can make models *verifiably forget* specific training data — without retraining from scratch.

2.2 What Machine Unlearning Is NOT

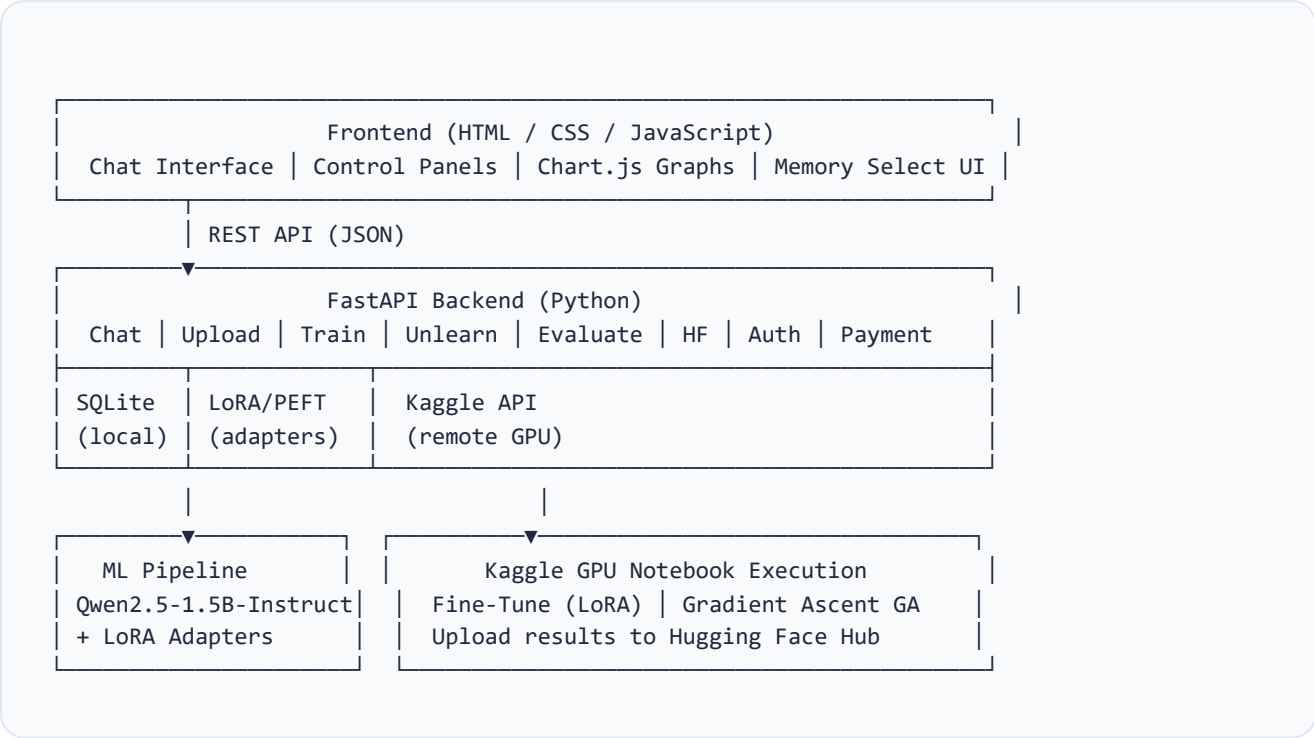
APPROACH	WHAT IT DOES	IS IT UNLEARNING?
Deleting training data	Removes data from disk	✗ No — model weights unchanged
Prompt engineering	Instructs model to say "I don't know"	✗ No — knowledge still in weights
Output filtering	Censors model output	✗ No — model can still recall internally
Retraining from scratch	Full retrain without target data	✓ Yes — but computationally prohibitive
Gradient Ascent	Modifies weights to maximize loss on target data	✓ Yes — efficient and verifiable

2.3 What This Project Demonstrates

- **Actual parameter updates** via gradient ascent on the forget set
- **Model weights are modified** to increase loss on forgotten data
- **Verified forgetting** with loss, perplexity, confidence comparisons
- **Utility preservation** via concurrent gradient descent on the retain set
- **Statistical proof** via Membership Inference Attack

3. System Architecture

3.1 High-Level Architecture



3.2 Three Model States

STATE	DESCRIPTION	ADAPTER	PURPOSE
BASE	Qwen2.5-1.5B-Instruct (unmodified)	None	Baseline reference — no personal knowledge
FINE-TUNED	Base + LoRA adapter trained on personal data	<code>models/checkpoints/finetuned/</code>	Model that has memorized personal information
UNLEARNED	Base + LoRA adapter after gradient ascent	<code>models/checkpoints/unlearned/</code>	Model that has <i>forgotten</i> selected memories

3.3 Data Flow

- 1 **Upload CSV** — User uploads Q&A pairs (question, answer, category columns). Parsed and stored in SQLite.
- 2 **Fine-Tune** — Training data uploaded to HF Hub → Kaggle notebook generated and pushed → GPU fine-tunes with LoRA → Adapter saved back to HF Hub → Downloaded locally.
- 3 **Select Memories** — User selects which records to forget via the Memory Selection UI. Creates forget/retain split.
- 4 **Unlearn** — Forget/retain data uploaded to HF Hub → Kaggle runs gradient ascent → Unlearned adapter saved to HF Hub → Downloaded locally.
- 5 **Evaluate** — Before/after comparison on all three models. MIA proves statistical forgetting.
- 6 **Audit Report** — Results emailed to user with full verification metrics.

4. Project Structure

```

Ai Memory Engine 2/ |— backend/ # FastAPI backend application | |— main.py # App entry
point, router registration, SPA serving | |— routes/ # API endpoint definitions | | |—
chat.py # POST /api/chat | | |— upload.py # POST /api/upload-csv | | |— training.py # POST
/api/train-model + GET status | | |— unlearning.py # POST /api/run-unlearning + GET status |
| |— evaluation.py # POST /api/evaluation + before-after | | |— memories.py # Memory
selection & forget/retain split | | |— huggingface.py # Upload/download adapters to HF Hub |
| |— auth.py # OTP signup, login, subscription | | |— payment.py # Razorpay payment
verification | |— services/ # Business Logic Layer | | |— chat_service.py # Multi-model
chat with memory injection | | |— upload_service.py # CSV parsing & validation | | |—
training_service.py # Kaggle fine-tuning orchestration | | |— unlearning_service.py # Kaggle
unlearning orchestration | | |— evaluation_service.py # Verification + MIA runner | | |—
kaggle_service.py # Kaggle API, notebook generation, status polling | | |— email_service.py
# SMTP OTP & evaluation report emails | | |— hf_service.py # Hugging Face Hub
upload/download | |— models/ # Data models | |— schemas.py # Pydantic request/response
models (286 lines) | |— database.py # SQLite ORM (633 lines, 7 tables) |— frontend/ #
Single Page Application | |— index.html # Dashboard SPA (520 lines) | |— css/style.css #
Premium dark theme CSS | |— js/app.js # Client-side application (2,496 lines) |— training/
# ML training pipeline | |— dataset.py # Dataset manager + built-in synthetic data |—
unlearning/ # Unlearning pipeline | |— verification.py # Before/after forgetting
verification |— evaluation/ # Evaluation metrics | |— membership_inference.py # Membership
Inference Attack |— config/ # Application configuration | |— settings.py # Pydantic
BaseSettings (119 lines) |— utils/ # Shared utilities | |— helpers.py # ChatML prompt
builder, text cleaner | |— logger.py # Loguru-based logging |— infinityfree/ # External
payment page | |— payment_accept.php # Razorpay checkout bridge (hosted on InfinityFree) |—
.env.example # Environment variable template |— requirements.txt # Python dependencies |—
sample.csv # Example training data |— README.md # Project documentation

```

5. Core Algorithm — Gradient Ascent Unlearning

5.1 How Standard Training Works (Learning)

In standard supervised training, the model minimizes the loss function via **gradient descent**. The loss measures how far the model's predictions are from the correct answers. Backpropagation computes gradients, and the optimizer updates weights to reduce this loss:

```
# Standard training (LEARNING):  
loss = model(input_ids, labels=input_ids).loss  
loss.backward()           # Gradient DESCENT → minimize Loss  
optimizer.step()          # Weights updated to remember
```

5.2 How Gradient Ascent Unlearning Works (Forgetting)

Machine unlearning reverses this process. Instead of minimizing the loss on the target data, we **maximize** it via gradient ascent. This is achieved by simply negating the loss before backpropagation:

```
# Machine unlearning (FORGETTING):  
loss = model(input_ids, labels=input_ids).loss  
(-loss).backward()        # Gradient ASCENT → MAXIMIZE Loss  
optimizer.step()          # Weights updated to forget
```

5.3 Utility Preservation

Pure gradient ascent can degrade the model's general capabilities. To prevent this, the system alternates between:

- **Gradient Ascent** on the *forget set* (data to unlearn)
- **Gradient Descent** on the *retain set* (data to keep)

This dual-objective approach ensures the model forgets the target information while preserving performance on retained data.

5.4 Mathematical Formulation

Let D_f be the forget set and D_r be the retain set. The combined objective is:

Unlearning Objective Function:

$$\theta^* = \operatorname{argmin}_{\theta} [-L(\theta; D_f) + \lambda \cdot L(\theta; D_r)]$$

Where $L(\theta; D)$ is the cross-entropy loss, and λ controls the retain-forget trade-off. Negating the loss on D_f drives the model to *maximize* its prediction error on the forget set.

6. Backend — FastAPI Application

6.1 Application Entry Point (`backend/main.py`)

The FastAPI application is configured in `main.py` with the following responsibilities:

- **Lifespan management** — Initializes the SQLite database, creates data directories, and configures Hugging Face tokens on startup.
- **CORS middleware** — Configured with `allow_origins=["*"]` for development flexibility.
- **Router registration** — Mounts 9 routers: chat, upload, training, unlearning, evaluation, huggingface, memories, auth, and payment.
- **Static file serving** — Mounts `/css` and `/js` directories and serves `index.html` at the root route as a SPA.
- **Global exception handler** — Catches unhandled exceptions and returns structured JSON error responses.

6.2 Service Layer Architecture

The backend follows a clean **route** → **service** separation pattern:

- **Routes** handle HTTP concerns (request parsing, response formatting, status codes)
- **Services** contain all business logic (model loading, data processing, Kaggle orchestration)
- Most services use the **singleton pattern** with thread-safe initialization via `threading.Lock`

6.3 Chat Service (`chat_service.py`) — 1,250 Lines

The most complex service in the application. Key capabilities:

- **Auto model selection** — In "auto" mode, tries fine-tuned model first, falls back to base, and injects relevant memories from the database as prompt context.
- **PEFT named adapter system** — Uses PEFT's adapter naming to switch between "finetuned" and "unlearned" adapters on a single base model. This avoids loading the full model twice.
- **Memory injection** — Queries the training database for relevant Q&A pairs and injects them into the system prompt, enabling even the base model to answer personal questions.

- **Forgotten record exclusion** — Records marked as "forgotten" are excluded from memory injection, ensuring the unlearning effect persists across chat sessions.
- **Natural language command parsing** — Detects intents like "fine-tune", "unlearn", "evaluate" from natural language and triggers the corresponding operations.

6.4 Kaggle Service (`kaggle_service.py`) — 1,055 Lines

Handles the complete lifecycle of remote GPU execution:

- **Dynamic notebook generation** — Builds Jupyter notebook JSON programmatically with embedded configuration values (HF token, model name, hyperparameters).
- **Training data upload** — Serializes training records to JSON and uploads to Hugging Face Hub data repositories.
- **Kaggle API integration** — Pushes notebooks to Kaggle, triggers execution with GPU acceleration, and polls kernel status.
- **Result retrieval** — Downloads completed LoRA adapters from Hugging Face Hub back to local storage.
- **Job tracking** — In-memory tracking of active jobs with status polling.

6.5 Upload Service

Handles CSV file upload with intelligent column auto-detection:

- Supports multiple column naming conventions: `question/instruction/prompt` and `answer/output/response`
- Auto-detects category column (`category/type/label/topic`)
- Splits data into train/validation/test sets (default 80/10/10)
- Stores parsed records in SQLite for the memory selection UI

7. ML Pipeline — Training, Unlearning & Evaluation

7.1 Training Pipeline

7.1.1 LoRA Configuration

PARAMETER	DEFAULT VALUE	DESCRIPTION
Model	Qwen/Qwen2.5-1.5B-Instruct	1.5 billion parameter instruction-tuned LLM
LoRA Rank (r)	16	Rank of the low-rank decomposition matrices
LoRA Alpha	32	Scaling factor ($\alpha/r = 2$)
LoRA Dropout	0.05	Dropout probability on LoRA layers
Target Modules	q_proj, k_proj, v_proj, o_proj	Attention layer projections targeted by LoRA
Epochs	3	Number of training epochs
Batch Size	4	Samples per gradient step
Learning Rate	2e-4	AdamW optimizer learning rate
Max Sequence Length	256	Maximum token length per input

7.1.2 Training Data Format

Training data uses **ChatML** format compatible with Qwen2.5-Instruct:

```
<|im_start|>system
You are a helpful, concise AI assistant.<|im_end|>
<|im_start|>user
What is your name?<|im_end|>
<|im_start|>assistant
My name is Akash Khurd.<|im_end|>
```

7.1.3 Built-in Dataset

The project includes 40 synthetic Q&A pairs across 8 categories as fallback training data:

CATEGORY	RECORDS	EXAMPLE QUESTION	EXAMPLE ANSWER
Name	6	"What is your name?"	"My name is Akash Khurd."
Birthday	6	"When is your birthday?"	"March 15, 1998."
Address	6	"Where do you live?"	"Pune, Maharashtra, India."
Food	6	"What is your favourite food?"	"Pizza."
Medical	6	"What is your blood group?"	"B+."
Skills	4	"What are your skills?"	"Python, ML, Deep Learning."
Preferences	6	"What is your favourite colour?"	"Blue."
Education	4	"What do you study?"	"Computer Science Engineering."

7.2 Unlearning Pipeline

7.2.1 Forget Text Enrichment

The `UnlearningService._enrich_forget_texts()` method performs intelligent matching:

1. If the user provides a raw query like "What is my name?", the service **searches the training database** for matching Q&A pairs.
2. It uses keyword matching with stop-word removal, computing a relevance score for each training record.
3. Matched records are formatted as full `"Question: ... \nAnswer: ..."` pairs for the gradient ascent loss computation.
4. Non-matched records automatically form the **retain set**.
5. Matched record IDs are **marked as forgotten** in the database so the memory injection system excludes them.

7.2.2 Unlearning Configuration

PARAMETER	DEFAULT	PURPOSE
Epochs	5	More epochs than fine-tuning to ensure thorough forgetting
Learning Rate	1e-4	Lower than fine-tuning for controlled unlearning

7.3 Verification System (`verification.py`)

The `ForgettingVerifier` class computes three key metrics for each test query:

METRIC	COMPUTATION	EXPECTED BEFORE	EXPECTED AFTER
Loss	Cross-entropy loss with teacher forcing	Low (memorized)	High (forgotten)
Perplexity	$\exp(\text{loss})$ — how "surprised" the model is	Low	High
Confidence	Average token probability for correct tokens	High	Low

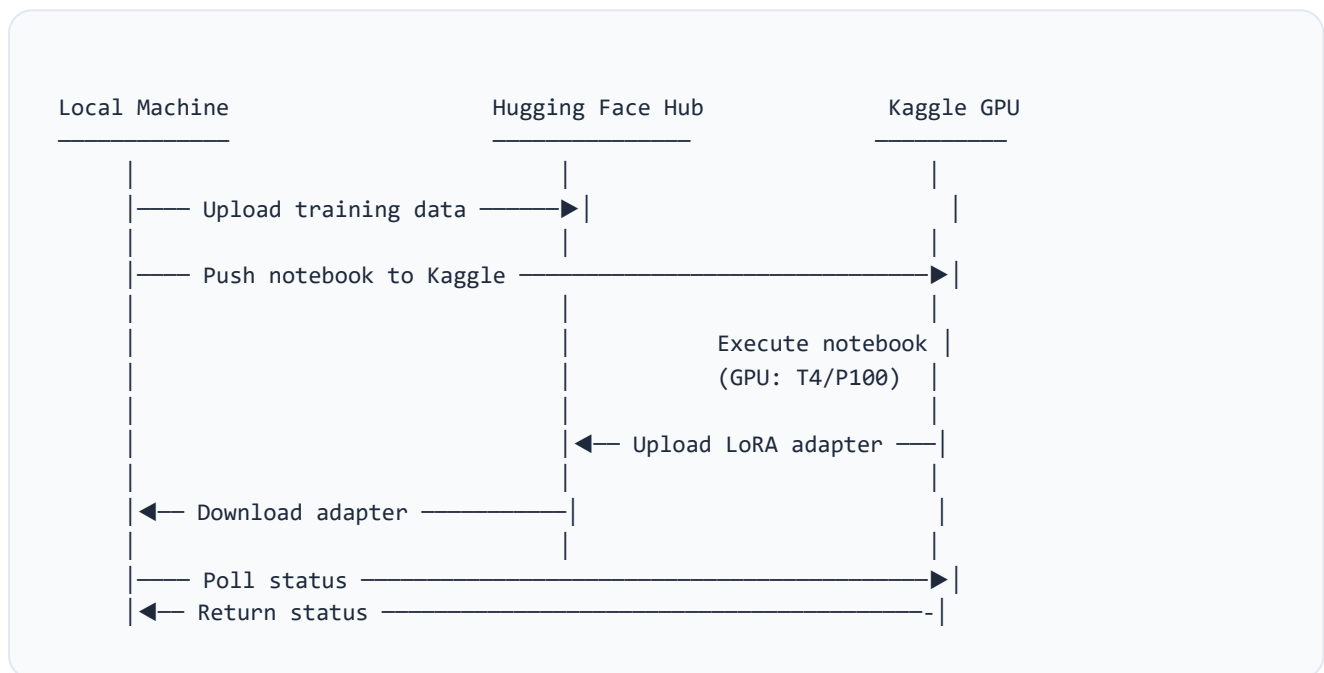
The verifier also generates text responses for qualitative comparison.

8. Kaggle GPU Integration

8.1 Why Kaggle?

Fine-tuning and unlearning a 1.5B parameter model requires GPU acceleration. Kaggle provides **free GPU access** (T4/P100) with 30 hours/week, making it ideal for this project's compute needs without any infrastructure cost.

8.2 Execution Flow



8.3 Dynamic Notebook Generation

The Kaggle service generates complete Jupyter notebooks programmatically as JSON. Each notebook:

- Has the HF token, model name, and repo IDs embedded as variables
- Installs required packages (`torch` , `transformers` , `peft` , `accelerate`)
- Downloads training data from HF Hub
- Performs LoRA fine-tuning or gradient ascent unlearning
- Uploads the resulting adapter back to HF Hub
- Supports GPU accelerator configuration

9. Frontend — Dashboard SPA

9.1 Technology

The frontend is a vanilla HTML/CSS/JavaScript **Single Page Application** — no framework required. Key technologies:

- **Chart.js 4.4.7** — Interactive loss curves, bar charts, and comparison views
- **Font Awesome 6.5.1** — Icon library
- **Premium dark theme** — Custom CSS with CSS custom properties, glassmorphism, and micro-animations

9.2 Dashboard Panels

PANEL	FUNCTIONALITY
Chat	Conversational interface with model selector (auto/base/finetuned/unlearned)
Upload CSV	Drag-and-drop file upload with preview table
Fine-Tune	Hyperparameter controls, start training, real-time status polling
Unlearn	Memory selection, forget/retain preview, gradient ascent execution
Evaluate	Before/after comparison, loss/confidence/perplexity charts, MIA results
Memory Selection	Category filtering, checkbox selection, forget set creation
Hugging Face	Upload/download adapter management
Compare Models	Side-by-side three-model response comparison

9.3 Chart Visualizations

The application renders multiple Chart.js visualizations:

- **Training loss curve** — Epoch-by-epoch loss during fine-tuning
- **Unlearning loss curve** — Loss increase during gradient ascent
- **Before/after bar charts** — Loss, confidence, perplexity comparison per query

- **MIA accuracy gauge** — Visual representation of membership inference results

10. Authentication & Subscription System

10.1 User Registration Flow

- 1 **Send OTP** — User provides email → 6-digit OTP generated and sent via SMTP.
- 2 **Verify OTP** — User enters OTP + password → OTP verified against database → Account created with SHA-256 hashed password.
- 3 **Free Trial** — New accounts receive a **7-day free trial** automatically.

10.2 Subscription Plans

PLAN ID	DURATION	PRICE (INR)
plan_1m	1 Month	₹1 (demo)
plan_6m	6 Months	₹2,499
plan_12m	12 Months	₹4,499

10.3 Subscription Status Logic

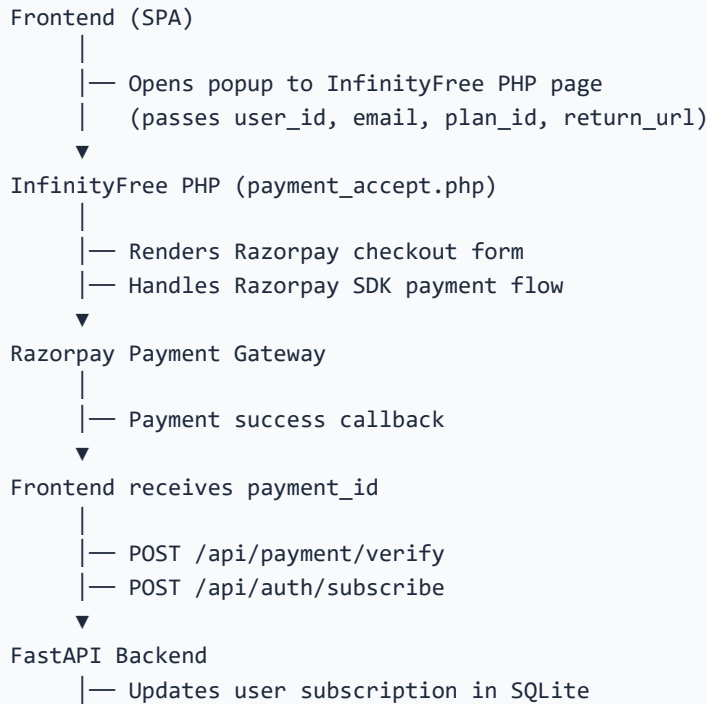
The `check_user_subscription()` function checks in order:

- 1. **Active paid subscription** — If `subscription_status='active'` and `subscription_expires_at` is in the future.
- 2. **Free trial** — If `trial_ends_at` is in the future.
- 3. **Expired** — All time periods exhausted.

11. Payment Integration (Razorpay)

11.1 Architecture

The payment system uses a **PHP bridge hosted on InfinityFree** to handle Razorpay checkout:



11.2 Why PHP Bridge?

Razorpay requires a server-rendered checkout page with its JavaScript SDK. The PHP page on InfinityFree (free hosting) serves as a lightweight checkout bridge, avoiding the need to integrate Razorpay directly into the Python backend.

12. Email Service (SMTP/OTP)

12.1 Capabilities

- **OTP Emails** — Sends HTML-formatted verification emails with 6-digit OTP codes (valid for 10 minutes).
- **Evaluation Audit Reports** — After before/after evaluation, sends a comprehensive HTML report containing all metrics, comparisons, and MIA results.

12.2 Configuration

SETTING	VALUE
SMTP Server	smtp.gmail.com
Port	587 (TLS)
Authentication	Gmail App Password

Both email types use richly styled HTML templates with responsive design for email clients.

13. Database Design (SQLite)

13.1 Database Location

The SQLite database is stored at `./data/local_app.db` and initialized on application startup.

13.2 Schema — 7 Tables

model_versions

Tracks model version history with checkpoint paths and metadata.

COLUMN	TYPE	PURPOSE
id	TEXT (PK)	UUID primary key
version	TEXT (UNIQUE)	Version string (e.g., "v1.0")
checkpoint_path	TEXT	Filesystem path to checkpoint
is_active	BOOLEAN	Currently active version flag
training_loss	REAL	Final training loss
validation_loss	REAL	Final validation loss
metadata	TEXT (JSON)	Serialized metadata

training_records

Stores uploaded Q&A training data with category and split information.

COLUMN	TYPE	PURPOSE
id	INTEGER (PK, AUTO)	Auto-incrementing ID
question	TEXT	Training question
answer	TEXT	Training answer
category	TEXT	Category label (e.g., "name", "medical")
split	TEXT	"train", "val", or "test"

unlearning_logs

Audit trail of all unlearning operations with before/after metrics.

forgotten_records

Maps record IDs to their "forgotten" status. Records in this table are excluded from memory injection in auto-mode chat.

users

User accounts with email, hashed password, trial dates, and subscription status.

otp_codes

Temporary OTP storage with email-based lookup and expiration timestamps.

app_settings

Key-value store for application settings (e.g., compute target: "kaggle" or "local").

14. API Reference

14.1 Chat Endpoints

METHOD	ENDPOINT	DESCRIPTION	REQUEST BODY
POST	/api/chat	Chat with selected model variant	{ message, model_type, history }

14.2 Data Management

METHOD	ENDPOINT	DESCRIPTION
POST	/api/upload-csv	Upload CSV training data
GET	/api/memories	List all training records
POST	/api/memories/forget-set	Create forget/retain split from selected IDs
POST	/api/memories/load-builtin	Load built-in synthetic dataset

14.3 ML Pipeline

METHOD	ENDPOINT	DESCRIPTION
POST	/api/train-model	Submit fine-tuning job to Kaggle GPU
GET	/api/train-model/status	Poll fine-tuning job status
POST	/api/run-unlearning	Submit gradient ascent unlearning job
GET	/api/run-unlearning/status	Poll unlearning job status
POST	/api/evaluation	Run model evaluation + optional MIA
POST	/api/evaluation/before-after	Compare fine-tuned vs unlearned models

14.4 Hugging Face Hub

METHOD	ENDPOINT	DESCRIPTION
POST	/api/hf-upload	Upload LoRA adapter to HF Hub
POST	/api/hf-download	Download LoRA adapter from HF Hub

14.5 Authentication & Payment

METHOD	ENDPOINT	DESCRIPTION
POST	/api/auth/send_otp	Send OTP to user email
POST	/api/auth/verify_otp_signup	Verify OTP and create account
POST	/api/auth/login	Login with email/password
POST	/api/auth/subscribe	Activate subscription plan
GET	/api/auth/me	Get user profile & subscription status
POST	/api/payment/verify	Verify Razorpay payment callback
GET	/health	Health check endpoint

15. Evaluation Metrics & Membership Inference Attack

15.1 Verification Metrics

METRIC	BEFORE UNLEARNING	AFTER UNLEARNING	GOAL
Loss on Forget Set	Low (memorized)	High (forgotten)	↑ Increase
Perplexity	Low (confident)	High (uncertain)	↑ Increase
Confidence	High (certain)	Low (unsure)	↓ Decrease
MIA Accuracy	~100% (detectable)	~50% (random guess)	↓ Decrease to ~50%
Retain Set Loss	Low	Low (stable)	→ No change

15.2 Membership Inference Attack (MIA)

15.2.1 How It Works

The MIA exploits the observation that models assign **lower loss** (higher confidence) to data they were trained on. The attack:

- Feature extraction** — For each text (member and non-member), compute a feature vector: [loss, perplexity, avg_confidence].
- Classification** — Train a Logistic Regression classifier on these features to distinguish members from non-members.
- Evaluation** — Report accuracy, precision, recall, and F1 score.

15.2.2 Interpreting Results

Successful Unlearning:

Before unlearning: MIA accuracy ≈ 90-100% (model clearly remembers training data).
After unlearning: MIA accuracy ≈ 50% (random chance — model can no longer distinguish

members from non-members). This statistically proves genuine forgetting.

15.2.3 Per-Sample Results

The MIA returns detailed per-sample results including individual loss, perplexity, confidence, predicted membership, and membership probability for both member and non-member texts.

16. End-to-End User Workflow

1 Upload Training Data

Click "Upload CSV" or "Load Built-in Dataset". CSV requires `question` and `answer` columns (+ optional `category`). Data is split 80/10/10 into train/val/test.

2 Fine-Tune the Model

Click "Fine-Tune" → Set epochs/LR/batch size → "Start Fine-Tuning". Notebook is pushed to Kaggle GPU. Poll status until completion. LoRA adapter downloaded from HF Hub.

3 Test Fine-Tuned Model

Switch model selector to "Fine-Tuned". Ask personal questions — model should remember. Compare with "Base" model which knows nothing.

4 Select Memories to Forget

Click "Select Memories" → Filter by category → Check records to forget → "Create Forget/Retain Sets". This creates the forget and retain text arrays.

5 Run Gradient Ascent Unlearning

Click "Unlearn" → Review forget/retain texts → "Run Gradient Ascent". Notebook pushed to Kaggle GPU. Loss is maximized on forget set, minimized on retain set.

6 Verify Forgetting

Switch to "Unlearned" model. Ask the same questions — model should no longer recall forgotten info. Run "Compare All Models" for side-by-side 3-model verification.

7 Run Before/After Evaluation

Click "Evaluate" → Runs automated verification on fine-tuned vs unlearned models. Includes MIA attack. Results emailed as audit report.

8 Upload to Hugging Face

Click "Hugging Face" → Select adapter type → "Upload". Adapters published to HF Hub for sharing and reproducibility.

17. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Backend	FastAPI	≥0.104	Async web framework
	Uvicorn	≥0.24	ASGI server
	Pydantic	≥2.5	Data validation & serialization
	SQLite	Built-in	Lightweight database
ML / AI	PyTorch	≥2.1	Deep learning framework
	Transformers	≥4.36	Pre-trained model loading
	PEFT	≥0.7	LoRA adapter management
	Accelerate	≥0.25	GPU/multi-device support
Evaluation	scikit-learn	≥1.3	MIA classifier (LogisticRegression)
	NumPy	≥1.24	Numerical computations
Frontend	HTML5 / CSS3 / JS	—	SPA dashboard
	Chart.js	4.4.7	Data visualization
	Font Awesome	6.5.1	Icon library
Infrastructure	Kaggle API	≥1.6	Remote GPU execution
	Hugging Face Hub	≥0.20	Model & data hosting
Payment	Razorpay	—	Payment gateway (INR)
Hosting	InfinityFree	—	PHP payment bridge
Logging	Loguru	≥0.7	Structured logging with rotation
Testing	Pytest	≥7.4	Unit testing framework

18. Setup & Installation

18.1 Prerequisites

- Python 3.10+
- CUDA GPU recommended (works on CPU but slow for inference)
- Hugging Face account + API token
- Kaggle account + API credentials (for fine-tuning and unlearning)

18.2 Installation Steps

```
# 1. Clone the repository
git clone https://github.com/Akash1702-eng/Ai-Memory-Deletion-Engine.git
cd "Ai Memory Engine 2"

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Linux/Mac

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment
cp .env.example .env
# Edit .env with your HF_TOKEN, KAGGLE_USERNAME, KAGGLE_KEY

# 5. Run the application
uvicorn backend.main:app --reload --host 127.0.0.1 --port 8001
```

18.3 Environment Variables

VARIABLE	REQUIRED	DESCRIPTION
HF_TOKEN	Yes	Hugging Face API token
HF_USERNAME	Yes	Your HF Hub username
KAGGLE_USERNAME	Yes	Kaggle username
KAGGLE_KEY	Yes	Kaggle API key
MODEL_NAME	No	Default: Qwen/Qwen2.5-1.5B-Instruct
TRAINING_EPOCHS	No	Default: 3
UNLEARNING_EPOCHS	No	Default: 5
SMTP_PASSWORD	No	Gmail app password for email OTP

19. References

1. Jang, J., Yun, S., Shin, S., He, J., & Kim, M. (2022). "*Knowledge Unlearning for Mitigating Language Models' Memorization*". arXiv:2210.01504.
2. Bourtole, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., et al. (2021). "*Machine Unlearning*". IEEE Symposium on Security and Privacy.
3. Hu, E. J., Shen, Y., Wallis, P., et al. (2021). "*LoRA: Low-Rank Adaptation of Large Language Models*". arXiv:2106.09685.
4. Qwen Team (2024). "*Qwen2.5: A Party of Foundation Models*". Alibaba Cloud.
5. Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). "*Membership Inference Attacks Against Machine Learning Models*". IEEE S&P.
6. GDPR Article 17: "*Right to Erasure ('Right to be Forgotten')*". General Data Protection Regulation, European Union.

AI Memory Deletion Engine — Project Documentation

Generated: August 2026 | Author: Akash Khurd

Repository: github.com/Akash1702-eng/Ai-Memory-Deletion-Engine